

V2X 스마트 지팡이 — 오늘의 개발 실행 계획서

2026.05.03 | Phase 2: 하드웨어 통합 · 무선 통신 구축 · 교통 시뮬레이션

팀원별 파트 분리 — 각자 오늘 무엇을, 어떤 순서로, 어떻게 하는가

0. 오늘의 전체 그림

0.1 지난 시간까지 완료한 것

담당자	완료 항목	의미
강현준 박중선	ESP32 환경 설정 + Blink 테스트	펌웨어 업로드 파이프라인 확립
강현준 박중선	ICM-20948 IMU 센서 연결·검증	가속도/각속도/지자기 데이터 수집 가능 확인
강현준 박중선	Neo-6M GPS 모듈 야외 연결	위성 좌표 수신 인터페이스 작동 확인
최민서	SUMO 기본 설치 + 샘플 실행 테스트	시뮬레이터 구동 환경 확보

0.2 오늘 각자 해야 할 핵심 한 줄 요약

담당자	오늘의 미션	완료 기준
박중선	브레드보드 위에 ESP32 + 전체 센서 물리 배선 완성	I2C 스캐너에서 IMU 주소 검출 + GPS 좌표 동시 출력
강현준	ESP32 2대 간 ESP-NOW 1:1 통신 성공	송신기 더미 데이터가 수신기 시리얼 모니터에 출력
최민서	netedit으로 주차장 진출입로 도로망 + 차량 배치 → 시뮬레이션 재생	sumo-gui에서 차량이 움직이는 화면 캡처

파트 A. 하드웨어 통합 조립

박중선

오늘의 목표: 지금까지 개별로 테스트한 센서들을 ESP32 하나에 전부 물리적으로 연결하여, 하나의 코드에서 IMU + GPS 데이터가 동시에 나오는 것을 확인한다.

A-1. 준비물 체크 0~10분

책상 위에 아래 부품을 꺼내고 하나씩 체크한다:

- ☐ ESP32 DevKit V1 — 1개
- ☐ 브레드보드 (830홀 이상) — 1개
- ☐ ICM-20948 IMU 모듈 — 1개
- ☐ Neo-6M GPS 모듈 — 1개
- ☐ 점퍼 와이어 (수-수, 암-수) — 10개 이상
- ☐ USB Micro-B 케이블 — 1개
- ☐ 노트북 (Arduino IDE 설치 완료)

절대 규칙: 모든 배선이 끝나기 전까지 USB 케이블을 꽂지 않는다. 전원이 들어간 상태에서 선을 만지면 합선으로 보드가 영구 파손된다.

A-2. ESP32 중앙 배치 + 전원 레일 구성 10~20분

① ESP32를 브레드보드에 꽂기

- 브레드보드 정중앙의 홈(채널)을 기준으로 ESP32를 수직 방향으로 꽂는다
- 핀이 양쪽으로 각각 한 줄씩 나뉘어야 한다 (한쪽에 물리면 안 됨)
- 핀이 휘어지지 않게 양쪽 끝을 동시에 살짝 누르며 삽입

② 전원 레일 연결 (혈관 만들기)

출발점	→	도착점	색상
ESP32의 3V3 핀	→	브레드보드 빨간색(+) 레일	빨간 점퍼선
ESP32의 GND 핀	→	브레드보드 파란색(-) 레일	검은 점퍼선

→ 이제 빨간 레일에 꽂으면 3.3V 전원, 파란 레일에 꽂으면 GND가 공급된다.

사용 금지 핀 목록 (외워둘 것):

- **GPIO 6~11**: 내부 플래시 메모리 연결 → 절대 사용 금지
- **GPIO 0, 2, 5, 12, 15**: 스트래핑 핀(부팅 모드 결정) → 가급적 회피
- **GPIO 1, 3**: USB 시리얼(UART0)용 → 센서 연결 금지

A-3. ICM-20948 IMU 센서 연결 (I2C) 20~35분

왜 I2C인가?

IMU는 I2C 통신을 쓴다. 선 2가닥(SDA, SCL)으로 여러 센서를 병렬 연결할 수 있다. 각 센서는 고유 주소(IMU: 0x69)를 가지고 있어 충돌이 없다.

배선 순서 (총 4가닥)

ICM-20948 핀	→	연결 대상	역할
VCC	→	빨간 레일 (+)	3.3V 전원 공급
GND	→	파란 레일 (-)	공통 그라운드
SDA	→	ESP32 GPIO 21	데이터 전송선
SCL	→	ESP32 GPIO 22	클럭(박자) 동기화선

배선 직후 확인 행동

선 4가닥이 모두 꽂혔으면, 각 선의 양쪽 끝을 손가락으로 살짝 눌러서 헐렁하지 않은지 체크한다. 브레드보드 접촉 불량에 I2C 에러의 70% 이상을 차지한다.

A-4. Neo-6M GPS 모듈 연결 (UART2) 35~50분

왜 UART인가?

GPS는 UART 시리얼 통신을 쓴다. 1대1 전용 통로이므로, 반드시 TX↔RX를 교차 연결해야 한다.

배선 순서 (총 4가닥)

Neo-6M 핀	→	연결 대상	역할
VCC	→	빨간 레일 (+)	전원 (3.3V)
GND	→	파란 레일 (-)	공통 그라운드
TX (GPS가 말하는 입)	→	ESP32 GPIO 16 (RX2)	GPS → ESP32 수신
RX (GPS가 듣는 귀)	→	ESP32 GPIO 17 (TX2)	ESP32 → GPS 전송

가장 흔한 실수: TX끼리, RX끼리 연결하면 통신이 절대 안 된다. 반드시 **교차!** GPS의 TX → ESP32의 RX(16번), GPS의 RX → ESP32의 TX(17번).

A-5. 전체 배선 최종 점검 50~60분

점검 체크리스트 (하나씩 손가락으로 짚으며 확인)

- ☐ ESP32의 GND → 파란 레일 연결됨
- ☐ ESP32의 3V3 → 빨간 레일 연결됨
- ☐ IMU의 VCC → 빨간 레일, GND → 파란 레일
- ☐ IMU의 SDA → GPIO 21, SCL → GPIO 22
- ☐ GPS의 VCC → 빨간 레일, GND → 파란 레일
- ☐ GPS의 TX → GPIO 16, GPS의 RX → GPIO 17
- ☐ 선끼리 꼬이거나 닿는 곳 없음
- ☐ 빨간 레일과 파란 레일이 직접 연결된 곳 없음 (합선 방지)

완성 배선 핀맵 요약표

ESP32 핀	연결 대상	통신 방식
3V3	빨간 레일 (+)	전원
GND	파란 레일 (-)	공통 그라운드
GPIO 21	IMU SDA	I2C 데이터
GPIO 22	IMU SCL	I2C 클럭
GPIO 16	GPS TX	UART2 수신
GPIO 17	GPS RX	UART2 송신

A-6. 전원 연결 + I2C 스캐너 테스트 60~75분

이제 비로소 USB를 꽂는다!

USB 케이블로 ESP32를 노트북에 연결한다. 다음 코드를 업로드하여 IMU가 I2C 버스에서 잡히는지 확인한다:

```
// [박종선] I2C 스캐너 - IMU 연결 확인용
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22); // SDA=21, SCL=22
  delay(1000);
  Serial.println("I2C 스캐너 시작...");
}

void loop() {
  int deviceCount = 0;
  for (byte addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) {
      Serial.print("  장치 발견! 주소: 0x");
      if (addr < 16) Serial.print("0");
      Serial.println(addr, HEX);
      deviceCount++;
    }
  }
  if (deviceCount == 0) {
    Serial.println("  장치 없음 - SDA/SCL 배선 재확인!");
  } else {
    Serial.print("총 ");
    Serial.print(deviceCount);
    Serial.println("개 장치 감지");
  }
  Serial.println("—————");
  delay(3000);
}
```

기대 출력

```
I2C 스캐너 시작...
장치 발견! 주소: 0x69
총 1개 장치 감지
—————
```

0x69가 뜨면: IMU 연결 성공. 다음 단계로 진행.

장치 없음이 뜨면: ① SDA↔SCL이 바뀌지 않았는지 ② VCC/GND가 제대로 꽂혔는지 ③ 브레드보드 접촉 불량인지 점검.

A-7. GPS + IMU 동시 데이터 출력 통합 코드 75~110분

```
// [박중선] IMU + GPS 통합 읽기 테스트
#include <Wire.h>
#include <ICM_20948.h>
#include <TinyGPSPlus.h>

ICM_20948_I2C imu;
TinyGPSPlus gps;
HardwareSerial gpsSerial(2); // UART2

void setup() {
  Serial.begin(115200);
  delay(1000);

  // IMU 초기화
  Wire.begin(21, 22);
  imu.begin(Wire, 1); // AD0=HIGH → 0x69
  if (imu.status != ICM_20948_Stat_0k) {
    Serial.println("[IMU] 연결 실패!");
  } else {
    Serial.println("[IMU] 연결 성공 (0x69)");
  }

  // GPS 초기화
  gpsSerial.begin(9600, SERIAL_8N1, 16, 17);
  Serial.println("[GPS] UART2 시작 (RX=16, TX=17)");
  Serial.println("===== 센서 통합 테스트 시작 =====");
}

void loop() {
  // --- IMU 읽기 ---
  if (imu.dataReady()) {
    imu.getAGMT();
    Serial.print("[IMU] Ax=");
    Serial.print(imu.accX() / 1000.0, 2);
    Serial.print(" Ay=");
    Serial.print(imu.accY() / 1000.0, 2);
    Serial.print(" Az=");
    Serial.print(imu.accZ() / 1000.0, 2);
    Serial.print(" | Gx=");
    Serial.print(imu.gyrX(), 1);
    Serial.print(" Gy=");
    Serial.print(imu.gyrY(), 1);
    Serial.print(" Gz=");
    Serial.println(imu.gyrZ(), 1);
  }

  // --- GPS 읽기 ---
  while (gpsSerial.available() > 0) {
    gps.encode(gpsSerial.read());
  }

  if (gps.location.isUpdated()) {
    Serial.print("[GPS] 위도=");
  }
}
```

```

Serial.print(gps.location.lat(), 6);
Serial.print(" 경도=");
Serial.print(gps.location.lng(), 6);
Serial.print(" 위성수=");
Serial.println(gps.satellites.value());
}

Serial.println("_____");
delay(500);
}

```

기대 출력 (야외에서 GPS 수신 시)

```

[IMU] Ax=0.03 Ay=-0.01 Az=-1.00 | Gx=0.2 Gy=-0.5 Gz=0.1
[GPS] 위도=37.456312 경도=126.952034 위성수=7

```

GPS가 안 잡히면? GPS는 실내에서 위성 신호를 못 받는다. 창가나 야외로 이동하여 테스트하고, 첫 위치 잡기 (Cold Start)에 1~3분이 걸린다. 기다리는 동안 IMU 값이 정상 출력되면 배선은 성공이다.

A-8. 오늘 완료 기준 체크리스트 ~120분

- ☐ I2C 스캐너에서 0x69 주소 검출됨
- ☐ IMU 가속도(Ax, Ay, Az) 값이 실시간으로 변함
- ☐ 보드를 기울이면 가속도 값이 바뀌는 것을 확인함
- ☐ GPS 위도/경도가 시리얼에 출력됨 (야외 테스트)
- ☐ 두 센서가 동시에 한 ESP32에서 충돌 없이 동작함
- ☐ 배선 사진 촬영 완료 (기록용)

파트 B. ESP-NOW 무선 통신 구축 강현준

오늘의 목표: ESP32 2대를 사용하여 ESP-NOW 1:1 통신을 성공시킨다. 송신기에서 보낸 더미 데이터(GPS 좌표, 속도, 위험등급)가 수신기의 시리얼 모니터에 그대로 출력되는 것을 확인한다.

B-1. 준비물 체크 0~5분

- ☐ ESP32 DevKit — **2개** (1개: 송신기, 1개: 수신기)
- ☐ USB 케이블 — 2개 (각각 노트북에 연결)
- ☐ LED 1개 + 220Ω 저항 1개 (수신기 확인용, 없으면 내장 LED 사용)
- ☐ 노트북 (Arduino IDE)
- ☐ 포스트잇 or 마스킹 테이프 (보드에 "송신기" "수신기" 라벨 부착)

보드 구분이 핵심: 두 보드를 동시에 PC에 꽂으면 COM 포트가 2개 뜬다. 어떤 포트가 어떤 보드인지 헷갈리면 코드가 엉뚱한 보드에 올라간다. 반드시 포스트잇으로 "송신" "수신"을 보드에 붙여둘 것.

B-2. 수신기 MAC 주소 확인 5~15분

① 수신기 ESP32만 PC에 연결한다

② 아래 코드를 업로드한다

```
// [강현준] B-2: MAC 주소 확인 코드
// "수신기"라고 라벨 붙인 ESP32에 업로드
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  delay(1000);
  WiFi.mode(WIFI_STA);
  Serial.println("=====");
  Serial.print("이 ESP32의 MAC 주소: ");
  Serial.println(WiFi.macAddress());
  Serial.println("=====");
}

void loop() {}
```


③ 시리얼 모니터 열기 (115200 baud)

④ 출력된 MAC 주소를 메모장에 정확히 적기

예시 출력:

=====

이 ESP32의 MAC 주소: C8:F0:9E:52:78:A4

=====

⑤ MAC 주소를 코드용 배열로 변환해준다

C8:F0:9E:52:78:A4

→ uint8_t receiverMAC[] = {0xC8, 0xF0, 0x9E, 0x52, 0x78, 0xA4};

콜론(:)을 0x로 바꾸고 쉼표로 구분하면 된다. 한 글자라도 틀리면 통신 불가.

B-3. V2X 구조체 작성 15~25분

새 아두이노 스케치 2개를 만든다: **Sender.ino**, **Receiver.ino**

양쪽 파일의 맨 위에 완전히 동일한 구조체를 붙여넣는다:

```
// [강현준] B-3: V2X 데이터 구조체
// Sender.ino와 Receiver.ino 양쪽 모두 최상단에 동일하게 선언

#define NODE_CANE      0x01  // 지팡이
#define NODE_VEHICLE  0x02  // 차량
#define NODE_SIGNAL    0x03  // 신호등

#define RISK_SAFE      0      // 안전
#define RISK_CAUTION   1      // 주의
#define RISK_WARNING   2      // 경고
#define RISK_DANGER    3      // 위험

typedef struct v2x_message {
    uint8_t  node_type;        // 어떤 노드가 보냈는가
    uint8_t  risk_level;       // 위험 등급
    float    latitude;         // GPS 위도
    float    longitude;        // GPS 경도
    float    speed_mps;        // 이동 속도 (m/s)
    float    heading_deg;      // 방향각
    float    accel_x;          // IMU 가속도 X
    float    accel_y;          // IMU 가속도 Y
    float    accel_z;          // IMU 가속도 Z
    uint32_t timestamp_ms;     // 전송 시각
    uint8_t  seq_num;          // 패킷 번호 (유실 감지)
} v2x_message_t;
```

왜 구조체가 동일해야 하는가? 수신기는 받은 바이트 덩어리를 구조체 형태로 다시 해석한다. 송신기와 수신기의 구조체 필드 순서나 타입이 한 개라도 다르면, 위도 자리에 속도가 들어가는 등 데이터가 전부 꼬인다.

B-4. 송신기(Sender) 전체 코드 작성 25~50분

"송신기" 라벨이 붙은 ESP32에 업로드할 코드:

```
// [강현준] B-4: ESP-NOW 송신기 전체 코드 (Sender.ino)
#include <esp_now.h>
#include <WiFi.h>

// *** 여기에 B-2에서 확인한 수신기 MAC 주소 입력 ***
uint8_t receiverMAC[] = {0xC8, 0xF0, 0x9E, 0x52, 0x78, 0xA4};

// --- 구조체 (B-3에서 만든 것과 동일) ---
#define NODE_VEHICLE 0x02
#define RISK_SAFE 0

typedef struct v2x_message {
    uint8_t node_type;
    uint8_t risk_level;
    float latitude;
    float longitude;
    float speed_mps;
    float heading_deg;
    float accel_x;
    float accel_y;
    float accel_z;
    uint32_t timestamp_ms;
    uint8_t seq_num;
} v2x_message_t;

v2x_message_t outgoing;
uint8_t seq = 0;

// 전송 결과 콜백 - 보내고 나서 성공/실패 알림
void onSent(const uint8_t *mac, esp_now_send_status_t status) {
    Serial.print("[TX] #");
    Serial.print(seq);
    Serial.println(status == ESP_NOW_SEND_SUCCESS
        ? " → 성공 ✓" : " → 실패 ✗");
}

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("=== 송신기(차량 노드) 시작 ===");

    // 1) Wi-Fi 안테나 켜기
    WiFi.mode(WIFI_STA);
    Serial.print("송신기 MAC: ");
    Serial.println(WiFi.macAddress());

    // 2) ESP-NOW 초기화
    if (esp_now_init() != ESP_OK) {
        Serial.println("ESP-NOW 초기화 실패!");
        ESP.restart();
    }
}
```

```

}

// 3) 전송 결과 콜백 등록
esp_now_register_send_cb(onSent);

// 4) 수신기를 Peer(친구)로 등록
esp_now_peer_info_t peer = {};
memcpy(peer.peer_addr, receiverMAC, 6);
peer.channel = 0;
peer.encrypt = false;
if (esp_now_add_peer(&peer) != ESP_OK) {
    Serial.println("Peer 등록 실패!");
}

Serial.println("초기화 완료. 전송 시작...");
}

void loop() {
    // 5) 더미 데이터 채우기 (나중에 실제 센서로 교체)
    outgoing.node_type    = NODE_VEHICLE;
    outgoing.risk_level   = RISK_SAFE;
    outgoing.latitude     = 37.4563;
    outgoing.longitude    = 126.9520;
    outgoing.speed_mps     = 8.33;      // 약 30km/h
    outgoing.heading_deg  = 180.0;     // 남쪽
    outgoing.accel_x       = 0.05;
    outgoing.accel_y       = -0.02;
    outgoing.accel_z       = -9.81;
    outgoing.timestamp_ms = millis();
    outgoing.seq_num       = seq++;

    // 6) 전송!
    esp_now_send(receiverMAC,
                  (uint8_t *)&outgoing,
                  sizeof(outgoing));

    delay(100); // 100ms = 10Hz (초당 10번)
}

```

코드 작성 시 반드시 확인할 것

- **receiverMAC[]** — B-2에서 적어둔 주소가 정확한가? (가장 중요)
- **구조체** — Receiver.ino에 넣을 것과 필드 순서, 타입이 100% 동일한가?
- **delay(100)** — 너무 빠르면(10ms 이하) 수신기가 못 따라옴, 100ms가 안정적

B-5. 수신기(Receiver) 전체 코드 작성 50~75분

"수신기" 라벨이 붙은 ESP32에 업로드할 코드:

```
// [강현준] B-5: ESP-NOW 수신기 전체 코드 (Receiver.ino)
#include <esp_now.h>
#include <WiFi.h>

#define LED_PIN 2 // ESP32 내장 LED

// --- 구조체 (송신기와 완전히 동일!) ---
#define NODE_VEHICLE 0x02
#define RISK_SAFE 0
#define RISK_CAUTION 1
#define RISK_WARNING 2
#define RISK_DANGER 3

typedef struct v2x_message {
    uint8_t node_type;
    uint8_t risk_level;
    float latitude;
    float longitude;
    float speed_mps;
    float heading_deg;
    float accel_x;
    float accel_y;
    float accel_z;
    uint32_t timestamp_ms;
    uint8_t seq_num;
} v2x_message_t;

v2x_message_t incoming;
uint8_t prevSeq = 0;
uint32_t recvCount = 0;
uint32_t lostCount = 0;

// ★ 핵심: 수신 콜백 함수 ★
// 데이터가 도착하면 자동으로 이 함수가 실행됨
void onDataRecv(const esp_now_recv_info_t *info,
                const uint8_t *data, int len) {
    // 받은 바이트 → 구조체로 변환
    memcpy(&incoming, data, sizeof(incoming));
    recvCount++;

    // 패킷 유실 감지
    if (recvCount > 1) {
        uint8_t expected = prevSeq + 1;
        if (incoming.seq_num != expected) {
            uint8_t gap = incoming.seq_num - expected;
            lostCount += gap;
            Serial.print(" △ 패킷 ");
            Serial.print(gap);
            Serial.println("개 유실!");
        }
    }
}
```

```

}
prevSeq = incoming.seq_num;

// 수신 내용 출력
Serial.println("_____");
Serial.print("[RX] #");
Serial.print(incoming.seq_num);
Serial.print(" | 노드: ");
Serial.println(incoming.node_type == NODE_VEHICLE
               ? "차량" : "기타");

Serial.print(" 좌표: (");
Serial.print(incoming.latitude, 4);
Serial.print(", ");
Serial.print(incoming.longitude, 4);
Serial.println(")");

Serial.print(" 속도: ");
Serial.print(incoming.speed_mps * 3.6, 1);
Serial.print(" km/h | 방향: ");
Serial.print(incoming.heading_deg, 0);
Serial.println("°");

Serial.print(" 위험등급: ");
Serial.print(incoming.risk_level);
Serial.print(" | 총수신: ");
Serial.print(recvCount);
Serial.print(" | 유실: ");
Serial.println(lostCount);

// LED 깜빡 (수신 확인)
digitalWrite(LED_PIN, HIGH);
delay(30);
digitalWrite(LED_PIN, LOW);
}

void setup() {
  Serial.begin(115200);
  delay(1000);
  pinMode(LED_PIN, OUTPUT);
  Serial.println("=== 수신기(지팡이 노드) 시작 ===");

  WiFi.mode(WIFI_STA);
  Serial.print("수신기 MAC: ");
  Serial.println(WiFi.macAddress());

  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW 초기화 실패!");
    ESP.restart();
  }

  // 콜백 등록 = 자동 문열림 시스템
  esp_now_register_recv_cb(onDataRecv);

  Serial.println("수신 대기 중...");
}

```

```
void loop() {  
  // 수신기의 loop()는 비어있어도 됨  
  // 모든 처리는 콜백에서 자동 실행  
  delay(1000);  
}
```

B-6. 업로드 및 통신 테스트 실행 75~100분

업로드 순서 (이 순서가 중요하다!)

① 수신기를 먼저 업로드

수신기 ESP32의 COM 포트를 선택 → Receiver.ino 업로드 → 시리얼 모니터 열기 → "수신 대기 중..." 확인 → 시리얼 모니터를 켜둔 채로 유지

② 송신기를 나중에 업로드

송신기 ESP32의 COM 포트를 선택 (Arduino IDE를 하나 더 열거나 포트만 변경) → Sender.ino 업로드 → 시리얼 모니터 열기

③ 양쪽 시리얼 모니터 동시 관찰

송신기 화면: "[TX] #0 → 성공 ✓"가 반복되는지

수신기 화면: "[RX] #0 | 노드: 차량" + 좌표/속도 데이터가 뜨는지

성공 시 수신기 시리얼 모니터 출력 예시

```
=== 수신기(지팡이 노드) 시작 ===  
수신기 MAC: C8:F0:9E:52:78:A4  
수신 대기 중...
```

```
[RX] #0 | 노드: 차량  
좌표: (37.4563, 126.9520)  
속도: 30.0 km/h | 방향: 180°  
위험등급: 0 | 총수신: 1 | 유실: 0
```

```
[RX] #1 | 노드: 차량  
좌표: (37.4563, 126.9520)  
속도: 30.0 km/h | 방향: 180°  
위험등급: 0 | 총수신: 2 | 유실: 0
```

B-7. 안 될 때 — 트러블슈팅 문제 발생 시

증상	원인 (90%)	해결법
송신기: "전송 실패 x" 반복	MAC 주소 오 타	receiverMAC[] 배열 값을 B-2 출력과 한 바이트씩 대조
송신기: "성공 ✓"인데 수신기에 아무것도 안 뜸	WiFi 채널 불 일치	양쪽 모두 WiFi.mode(WIFI_STA)인지 확인. 한쪽이라도 AP 모드면 채널이 달라짐
수신기 데이터가 깨져서 숫자가 이상함	구조체 불일치	Sender.ino, Receiver.ino의 typedef struct 필드 순서·타 입이 100% 동일한지 비교
업로드 실패 "Failed to connect"	BOOT 버튼 미사용	"Connecting..." 뜨는 순간 ESP32의 BOOT 버튼을 꼭 누른 다
COM 포트가 안 보임	드라이버 미설 치	CP2102 또는 CH340 드라이버 검색하여 설치

B-8. 통신 성공 후 추가 실험 100~120분

실험 ①: 거리 테스트

수신기는 책상에 두고, 송신기를 들고 방 밖으로 나가본다:

거리	확인 사항
1m (바로 옆)	유실 0개, 안정적 수신
5m	유실 0개 유지되는지
10m (복도 끝)	유실률 기록
30m+ (야외)	통신 끊기는 거리 확인

실험 ②: 위험등급 바꿔보기

송신기의 **outgoing.risk_level** 값을 0→1→2→3으로 바꿔서 다시 업로드하고, 수신기에서 위험등급 숫자가 바뀌는지 확인한다. 이것이 확인되면 나중에 진동모터·부저 제어의 기초가 된다.

B-9. 오늘 완료 기준 체크리스트

- ☐ 수신기 MAC 주소를 정확히 기록함
- ☐ 송신기에서 "전송 성공 ✓"가 반복적으로 출력됨
- ☐ 수신기에서 좌표, 속도, 위험등급 데이터가 출력됨

- ☐ 패킷 번호(seq_num)가 연속적으로 증가함 (유실 없음)
- ☐ 내장 LED가 데이터 수신 때마다 깜빡임
- ☐ 시리얼 모니터 스크린샷 캡처 완료

파트 C. SUMO 교통 시뮬레이션 구축 최민서

오늘의 목표: netedit으로 주차장 진출입로 형태의 도로망을 직접 그리고, 차량 20대를 배치하여, sumo-gui에서 차량이 실제로 움직이는 시뮬레이션을 재생한다.

C-1. SUMO 설치 확인 0~5분

- ☐ SUMO가 설치되어 있다 (이전에 완료)
- ☐ 터미널/CMD에서 **sumo --version** 입력 → 버전 번호 출력됨
- ☐ netedit 실행 가능 (시작메뉴 또는 터미널에서 netedit 입력)

작업 폴더 생성

바탕화면 또는 원하는 위치에 폴더 생성:
V2X_SUMO_sim/
└─ (여기에 모든 파일을 저장할 것)

C-2. netedit으로 도로망 그리기 5~40분

시나리오: 건물 주차장 진출입로 + 보도

우리 프로젝트의 핵심 사고 다발 구간인 "건물 모퉁이 + 주차장 진출입로"를 가상으로 만든다.

단계 ①: netedit 실행

터미널 또는 시작메뉴에서 **netedit** 실행. 빈 화면이 나타난다.

단계 ②: 직선 보도(인도) 만들기

- 키보드 **E**를 누른다 → Edge(도로) 생성 모드 활성화
- 화면 왼쪽 빈 공간을 **클릭** → 첫 번째 교차점(Node) 생성
- 마우스를 오른쪽으로 쭉 끌어서 **클릭** → 두 번째 교차점 생성
- 두 점 사이에 도로(Edge)가 자동 생성된다
- 이 도로가 **보행자 도로(인도)**가 될 것이다

단계 ③: 수직으로 차량 도로 만들기 (진출입로)

- 첫 번째 도로의 **중간 지점 근처** 교차점(Node)을 클릭
- 마우스를 **아래쪽**으로 쭉 끌어서 클릭
- 보도와 수직으로 교차하는 차량 도로가 생성된다

4. 이 도로가 주차장에서 나오는 차량 진출입로를 의미

단계 ④: T자형 교차로 완성

1. 필요하면 차량 도로를 반대편(위쪽)으로도 하나 더 그려서 십자(+) 형태를 만든다
2. 교차점 근처를 마우스로 가까이 가면 자동으로 교차로가 인식됨

단계 ⑤: 차선 수 및 속도 설정

1. 키보드 **I** → Inspect 모드 전환
2. 생성한 도로(Edge)를 클릭
3. 왼쪽 속성 패널에서:
 - **numLanes**: 1 (1차선) 또는 2 (왕복)
 - **speed**: 8.33 (= 30km/h → m/s 환산값)
4. 보행자 도로의 speed는 1.39 (= 5km/h, 보행 속도)

단계 ⑥: 저장

File → Save Network → 파일명: **parking_lot.net.xml** → V2X_SUMO_sim 폴더에 저장

팁: 처음부터 복잡하게 만들지 않는다! T자형 도로 1개만 만들어도 오늘의 목표는 달성이다. 복잡한 도로망은 기본이 동작한 후에 확장한다.

C-3. 차량 배치 및 경로 설정 (Demand) 40~70분

방법 A: netedit GUI로 직접 배치 (권장)

단계 ①: Demand 모드 전환

netedit 상단 탭에서 "**Demand**" 클릭 (또는 단축키 **F3**)

단계 ②: 차량 타입 정의

1. 왼쪽 패널에서 "**Type**" 선택
2. "**Create**" 클릭
3. 속성 설정:
 - id: **car**
 - vClass: **passenger** (승용차)
 - color: **blue** (화면에서 구분하기 쉽게)
 - maxSpeed: **11.11** (= 40km/h)

단계 ③: 경로(Route) 그리기

1. 왼쪽 패널에서 "**Route**" 선택
2. 차량이 출발할 도로(Edge)를 **클릭** → 녹색 표시
3. 지나갈 교차로의 도로를 **순서대로 클릭**
4. 도착할 도로를 **마지막으로 클릭**
5. → 녹색 선으로 경로가 그려짐
6. **Enter** 눌러서 경로 확정

단계 ④: 차량 흐름(Flow) 생성

1. 왼쪽 패널에서 "**Flow**" (또는 "Vehicle") 선택
2. 방금 만든 경로를 선택
3. 속성 설정:
 - type: **car** (위에서 만든 타입)
 - begin: **0** (0초부터 시작)
 - end: **100** (100초까지)
 - number: **20** (총 20대 투입)

단계 ⑤: 저장

File → Save Demand Elements → 파일명: **parking_lot.rou.xml**

방법 B: 텍스트로 직접 작성 (GUI가 어려우면)

메모장을 열어 아래 내용을 직접 타이핑한다:

```
<!-- parking_lot.rou.xml -->
<routes>
  <!-- 차량 타입 정의 -->
  <vType id="car" vClass="passenger"
    color="blue" maxSpeed="11.11"
    accel="2.6" decel="4.5" length="5.0"/>

  <!-- 경로 정의 (Edge ID는 netedit에서 확인) -->
  <!-- Edge ID 예시: E0, E1, -E0 등 -->
  <route id="route_enter" edges="E1 E0"/>
  <route id="route_exit" edges="-E0 -E1"/>

  <!-- 차량 20대: 0~100초에 걸쳐 투입 -->
  <flow id="flow_in" type="car"
    route="route_enter"
    begin="0" end="100" number="10"/>

  <flow id="flow_out" type="car"
    route="route_exit"
    begin="10" end="100" number="10"/>
</routes>
```

Edge ID 확인법: netedit에서 Inspect 모드(키)로 도로를 클릭하면 왼쪽 패널 맨 위에 **id: E0** 같은 식으로 표시된다. 이 ID를 .rou.xml의 edges에 정확히 입력해야 한다. 반대 방향은 앞에 마이너스(-)를 붙인다: **-E0**

C-4. 통합 설정 파일(.sumocfg) 작성 70~80분

메모장을 열어 아래 내용을 타이핑하고 저장한다:

```
<!-- parking_lot.sumocfg -->
<configuration>
  <input>
    <!-- C-2에서 만든 도로망 파일 -->
    <net-file value="parking_lot.net.xml"/>
    <!-- C-3에서 만든 차량 파일 -->
    <route-files value="parking_lot.rou.xml"/>
  </input>

  <time>
    <begin value="0"/>
    <end value="200"/>
  </time>

  <output>
    <!-- AI 학습용 궤적 데이터 자동 저장 -->
    <fcd-output value="fcd_output.xml"/>
  </output>
</configuration>
```

파일 3개가 같은 폴더에 있는지 확인

```
V2X_SUMO_sim/
├─ parking_lot.net.xml      (도로망)
├─ parking_lot.rou.xml     (차량+경로)
└─ parking_lot.sumocfg     (통합 설정)
```

C-5. 시뮬레이션 실행! 80~100분

방법 ①: GUI로 실행 (시각적 확인)

1. 터미널/CMD를 열고 V2X_SUMO_sim 폴더로 이동
2. 아래 명령어 입력:

```
sumo-gui -c parking_lot.sumocfg
```

1. SUMO GUI 창이 열리면 **재생 버튼(▶)** 클릭
2. 차량이 너무 빠르게 지나가면 → 상단의 **Delay** 칸에 **100** 입력 (100ms 지연)
3. 화면을 마우스 휠로 확대/축소하여 차량이 도로를 따라 움직이는지 관찰

방법 ②: CLI로 실행 (데이터만 빠르게 추출)

```
sumo -c parking_lot.sumocfg
```

화면 없이 바로 fcd_output.xml이 생성된다 (빠름).

성공 기준: sumo-gui 화면에서 파란색 차량들이 도로를 따라 움직이고, 교차로에서 서로 양보하거나 정지하는 모습이 보이면 오늘의 목표 달성!

C-6. FCD 궤적 데이터 확인 100~115분

시뮬레이션이 끝나면 **fcd_output.xml** 파일이 생성된다. 메모장으로 열어본다:

```
<fcd-export>
  <timestep time="0.00">
    <vehicle id="flow_in.0" x="50.12" y="3.20"
      speed="8.33" angle="90.00"
      pos="10.00" lane="E1_0"/>
  </timestep>
  <timestep time="0.10">
    <vehicle id="flow_in.0" x="50.95" y="3.20"
      speed="8.33" angle="90.00"
      pos="10.83" lane="E1_0"/>
  </timestep>
  ...
</fcd-export>
```

이 데이터의 의미

필드	의미	AI에서의 활용
time	시뮬레이션 시각 (0.1초 단위)	시계열 데이터의 시간축
x, y	차량의 좌표 위치	Transformer 궤적 예측 입력
speed	순간 속도 (m/s)	급가속/급감속 이상 탐지
angle	진행 방향 (0~360°)	급회전 패턴 분석

향후 활용: 이 fcd_output.xml을 Python pandas로 파싱하면 차량별 궤적 DataFrame이 만들어지고, 이것이 Transformer 모델의 학습 데이터셋이 된다. 오늘은 이 파일이 정상 생성되는 것까지만 확인하면 된다.

C-7. 오늘 완료 기준 체크리스트

- ☐ netedit에서 T자형(또는 +자형) 도로망을 그렸다
- ☐ parking_lot.net.xml 파일이 저장되었다

- ☐ 차량 20대의 경로를 설정하였다
- ☐ parking_lot.rou.xml 파일이 저장되었다
- ☐ parking_lot.sumocfg 파일을 작성하였다
- ☐ sumo-gui에서 차량이 움직이는 화면을 확인하였다
- ☐ fcd_output.xml 파일이 생성되었다
- ☐ fcd_output.xml을 열어 차량 좌표 데이터가 들어있는 것을 확인하였다
- ☐ 화면 캡처 완료 (기록용)

전체 요약: 오늘 끝나면 우리 팀에 무엇이 생기는가

파트	담당	오늘 이전	오늘 이후
H/W	박중선	센서 각각 따로 동작 확인	하나의 ESP32에서 IMU+GPS 동시 데이터 출력
통신	강현준	ESP32 환경 설정만 완료	ESP32 2대 간 ESP-NOW 실시간 데이터 전송 성공
시뮬레이션	최민서	SUMO 설치 + 샘플 구동	자체 도로망 + 차량 시뮬레이션 + FCD 궤적 데이터 추출

다음 주 합류 지점 (3인 통합)

3인 통합 단계 미리보기

- ① 박중선이 만든 H/W 통합 보드의 센서 데이터를 → 강현준의 ESP-NOW 통신으로 전송
- ② 최민서의 SUMO FCD 데이터를 Python으로 파싱 → Transformer 학습 데이터셋 구축
- ③ TraCI를 통해 SUMO ↔ ESP32 실시간 연동 실험

오늘의 소통 규칙

- 각자 작업 중 막히는 부분은 즉시 카톡에 스크린샷과 함께 공유
- 각 파트 완료 시 체크리스트 기반으로 인증 사진/스크린샷을 팀 채팅에 올림
- 시리얼 모니터 출력 캡처, sumo-gui 화면 캡처, 배선 사진은 반드시 저장 (보고서용)